

Algoritmos e Programação de Computadores I

Strings em C

Prof. Marcelo Farias

Objetivos da Aula

Compreender string como vetor de char terminado em '\0'.
Manipular strings com <string.h> com segurança.
Preferir fgets; nunca usar gets.

Strings

- Vetor de caracteres terminado com o caractere especial de fim de string '\0'.
- Devem ser declaradas com um tamanho igual ao número de caracteres da string mais 1.
- Podem ser inicializadas com valores entre aspas duplas " ".

Obs.: C **não** tem um tipo de dados específico para strings.

Exemplo de Strings

// string dimensionada

```
char str1[14] = {'E', 'u', ' ', 'g', 'o', 's', 't', 'o', ' ', 'd', 'e', ' ', 'C', '\0'};
```

// string não-dimensionada

```
char str2[ ] = "Eu gosto de C";
```

// ponteiro para caractere

```
char *str3 = str1;
```

Manipulação de Strings

- C não suporta operações com strings.
- Manipulação de strings é feita com uso de funções da biblioteca `<string.h>`.

Funções de Manipulação de Strings

<code>memset(s1, c, n)</code>	Copia caractere c nas n primeiras posições de s1.
<code>strcpy(s1, s2)</code>	Copia s2 em s1.
<code>strcat(s1, s2)</code>	Concatena s2 no final de s1.
<code>strlen(s1)</code>	Retorna o tamanho de s1.
<code>strcmp(s1, s2)</code>	Retorna 0 se $s1 == s2$; < 0 se $s1 < s2$; > 0 se $s1 > s2$.
<code>strchr(s1, ch)</code>	Retorna a posição da primeira ocorrência de ch em s1.
<code>strstr(s1, s2)</code>	Retorna a posição da primeira ocorrência de s2 em s1.

memset

- Declaração: *memset(void *str, int c, size_t n);*
- Usada para preencher uma string com um caractere.

Exemplo de memset

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[80];
    memset(s1, 'a', 40); // preenche s1 com 40 caracteres a
    printf("%s\n", s1);

    return 0;
}
```


strcpy

- Declaração: *strcpy(char *dest, const char *src);*
- Usada para atribuir valores para uma string.

Exemplo de strcpy

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[80];
    strcpy(s1, "Isso é uma string!"); // copia a string em s1
    printf("%s\n", s1);

    return 0;
}
```

strcat

- Sintaxe: *strcat(char *dest, const char *src);*
- Usada para concatenar strings.

Exemplo de strcat

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[80] = "Isso é ";
    strcat(s1, "uma string"); // concatena a string com s1
    printf("%s\n", s1);

    return 0;
}
```

strlen

- Declaração: *strlen(const char *str);*
- Usada para calcular o tamanho da string (sem contar com **\0**).

Exemplo de strlen

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[80];
    fgets(s1, sizeof(s1), stdin);
    printf("tamanho da string: %ld", strlen(s1)); // calcula tamanho de s1

    return 0;
}
```

strcmp

- Declaração: *strcmp(const char *str1, const char *str2);*
- Usada para comparar duas strings pela ordem alfabética do conteúdo.

Exemplo de strcmp

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[80], s2[80];
    fgets(s1, sizeof(s1), stdin); fgets(s2, sizeof(s2), stdin);
    // comparação das strings s1 e s2
    printf("As strings são iguais? %d\n", strcmp(s1, s2));

    return 0;
}
```


strchr

- Declaração: *strchr(const char *str, int c);*
- Usada para localizar um caractere dentro de uma string.

Exemplo de strchr

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char s1[80];  
    int ch;  
    printf("Digite uma frase: ");  
    fgets(s1, sizeof(s1), stdin);  
    printf("Digite um caractere: ");  
    ch = getchar();  
    if (strchr(s1, ch) != NULL)  
        printf("caractere encontrado!\n");  
    printf("%s\n", strchr("alo", 'o'));  
    return 0;  
}
```

strstr

- Declaração: `char *strstr(const char *haystack, const char *needle);`

Usada para localizar uma string dentro de outra string.

Exemplo de strstr

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char s1[80], s2[80];  
    printf("Frase: ");  
    fgets(s1, sizeof(s1), stdin);  
    printf("Trecho: ");  
    fgets(s2, sizeof(s2), stdin);  
    if (strstr(s1, s2) != NULL)  
        printf("string encontrada!\n");  
    printf("%s\n", strstr("aqui alo", "alo"));  
    return 0;  
}
```

Matrizes de Strings

- Matriz bidimensional de caracteres.
- Uma dimensão indica o número de strings e a outra o tamanho de cada string.

Exemplo de Matriz de Strings

```
#define MAX 10
```

```
#define SIZE 61
```

```
char nomes[MAX][SIZE];
```

```
for (int i = 0; i < MAX; i++)  
    fgets(nomes[i], sizeof(nomes[i]), stdin);
```

```
for (int i = 0; i < MAX; i++)  
    printf("%s", nomes[i]);
```

Mapa da aula

Aula (teoria): Aula14 — Strings

Laboratório guiado: Pratica09

Atividade avaliativa: Atividade04 (strstr)

Sugestão de ritmo:

1) Conceitos nos slides 2) Prática no lab 3) Atividade sem 'receita'

Para estudar no livro

Livro-base: ANTONELLO, Ricardo. Linguagem C. 1. ed.

Leitura indicada: Cap. 8.3 — Strings (p. 102–112)

Exercícios do livro: Exercícios 8.4 (strings)

Revise o capítulo após a aula e antes da prática.

Desafio (8 min)

Leia frase e trecho com `fgets`; diga se o trecho ocorre (`strstr`). Remova o `'\n'` antes de comparar.

Trabalhe em dupla (instrução por pares).

Ao final, uma dupla compartilha a solução com a turma.

Quiz rápido (3 perguntas)

- 1) Por que a string termina com `'\0'`?
- 2) Diferença entre `strcpy` e `strcat`?
- 3) Por que `#define MAX 10; (com ;)` é erro?

Respostas no quadro / Mentimeter / Kahoot.

Referências

- SCHILDT, Herbert. C: Completo e Total. 3. ed. Pearson Makron Books, 1997.
- ANTONELLO, Ricardo. Linguagem C. 1. ed. Livro Digital, 2020.